



Asynchronous Programming

Concurrency

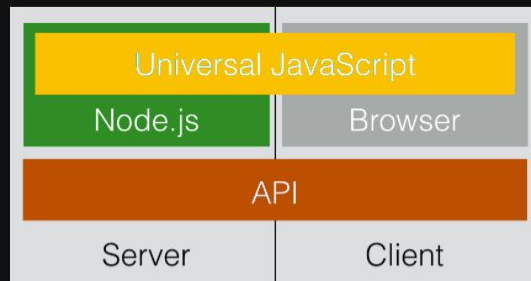


JavaScript is a single threaded, concurrent, non-blocking and asynchronous programming language

Execution Environment



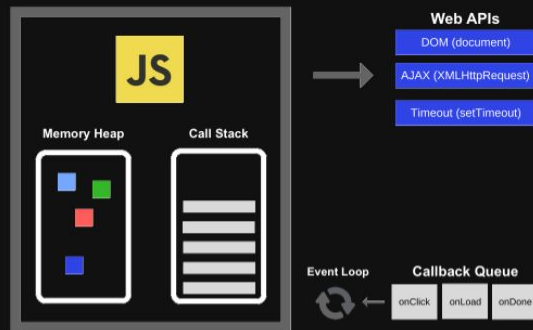
The JavaScript engine runs inside an execution environment such as the Browser or Node.js



This environment exposes many APIs to the JavaScript engine, such as the DOM or XMLHttpRequest (browser) or the Timers (both)

Concurrency Model

JavaScript has a concurrency model based on an event loop



This model is quite different from models in other languages like C and Java where concurrency is achieved mainly via parallel code execution using threads

Event Queue



Browser APIs expose operations that can run concurrently using a small number of threads

```
// to be called when data is available
function ajaxCallback() {
  console.log(this.responseText);
}

// to be called every 10 ms
function timerCallback() {
  console.log('tick');
}

setInterval(timerCallback, 10);

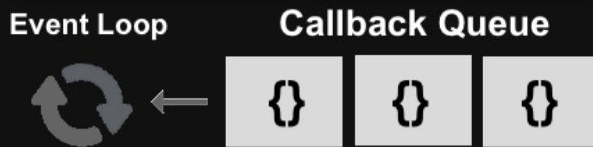
var ajax = new XMLHttpRequest();
ajax.addEventListener('load', ajaxCallback);
ajax.open('GET', 'https://api.github.com');
ajax.send();
```

Such operations result in a callback function to be executed by the JavaScript runtime

Event Loop



When the Call Stack is empty, the Event Loop takes the first callback from the Event Queue and places it at the top of the Call Stack for execution



```
console.log('start');
setTimeout(function() {
  console.log('setTimeout');
}, 0);
console.log('end');
// start, end, setTimeout
```

Each iteration of the event loop is called a **tick**

Rendering Engine



Browsers rendering engine executes in the same thread as the JavaScript runtime

Rendering events have higher priority for execution and will be fetched from the queue first

```
// what happens if we run this?  
while (true) {}
```

The event loop will not fetch anything from the queue **if the call stack is not empty**

Callback Functions



A function is called a **Callback** when it is passed as an argument to another function for later execution

Callbacks can be invoked synchronously or asynchronously:

```
// synchronous callback
function isOdd(num) {
  return num % 2;
}
var numbers = [1, 5, 8, 19, 232, 456];
var oddNumbers = numbers.filter(isOdd);
```

All *isOdd* callback invocations end
before the invoking *filter* function ends

```
// asynchronous callback
function ring() {
  console.log('RING!');
}

setInterval(ring, 1000);
```

The invoking *setInterval* function ends before the *ring*
callback is invoked

I/O Handling



To handle I/O operations, two different API models exists:

Synchronous and Blocking I/O

```
function doSthWithData(data) { ... }  
var data = getDataFromServer();  
doSthWithData(data);  
doSthElse();  
doSthElse();
```

var data = getDataFromServer()

IO: Request

Thread blocked

IO: Response

doSthWithData(data)

doSthElse();

doSthElse();

Asynchronous and Non-Blocking I/O

```
function doSthWithData(data) { ... }  
getDataFromServer(doSthWithData);  
doSthElse();  
doSthElse();
```

getDataFromServer(callback)

IO: Request

doSthElse();

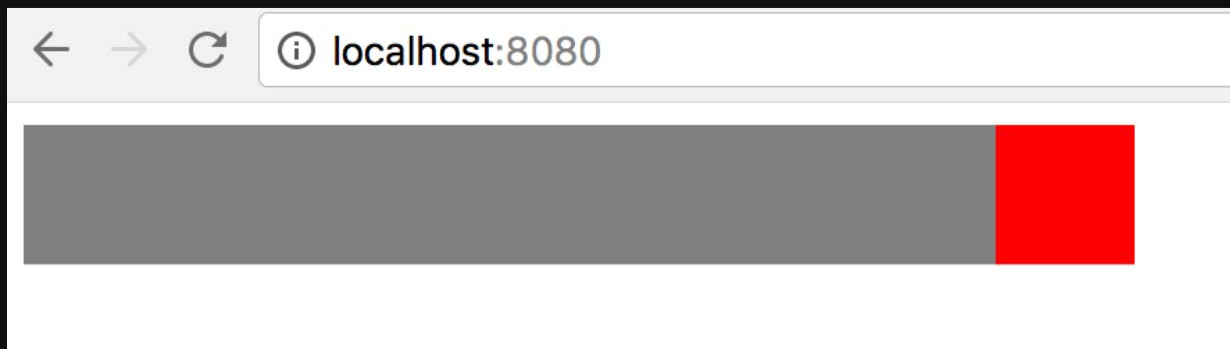
doSthElse();

IO: Response

callback: doSthWithData(data);

{ Hands On }

HTML Animation



JSON



JavaScript Object Notation is a lightweight data interchange format

It is based on a subset of JavaScript and widely used in web applications to exchange data between the client and the server

```
// a plain js object
var obj = { ticker: 'AAPL', name: 'Apple Inc' };

// serialize an object into a JSON string
var serObj = JSON.stringify(obj);

console.log(typeof obj); // 'object'
console.log(typeof serObj); // 'string'
console.log(serObj); // '{"ticker":"AAPL","name":"Apple Inc"}'
console.log(serObj.name); // undefined

// deserialize a JSON string into an object
var deSerObj = JSON.parse(serObj);

console.log(typeof deSerObj); // 'object'
console.log(deSerObj.name); // 'Apple Inc'
```

Introducing JSON

AJAX with Browser APIs



Allow loading data from the server without doing a page refresh

```
window.onload = function() {  
  
    fetchData(processResults);  
  
    function fetchData(cb) {  
        var ajax = new XMLHttpRequest();  
        ajax.addEventListener('load', cb);  
        ajax.open('GET', 'https://api.exchangeratesapi.io/latest');  
        ajax.send();  
    }  
  
    // asynchronous callback  
    function processResults(event) {  
  
        var results = JSON.parse(event.currentTarget.response);  
        var usd = results.rates.USD;  
  
        var container = document.getElementById('rate');  
        container.innerHTML = '<p>1 EUR = ' + usd + ' USD</p>';  
    }  
}
```

AJAX with jQuery



The jQuery library has a full suite of Ajax capabilities available, which wrap around the browser XMLHttpRequest

```
$(document).ready(function() {  
  
    fetchData(processResults);  
  
    function fetchData(cb) {  
        $.ajax({  
            url: 'https://api.exchangeratesapi.io/latest',  
            type: 'GET',  
            dataType: 'json',  
            success: cb  
        });  
    }  
  
    // asynchronous callback  
    function processResults(results) {  
        var usd = results.rates.USD;  
        var container = $('#rate');  
        container.html('<p>1 EUR = ' + usd + ' USD</p>');  
    }  
});
```

When using jQuery, there is no need to deserialize the JSON response

Async Error Handling



Handling errors inside async callbacks present some challenges

- Stack trace loses all the callback invocation context
- It is not possible to catch async callback error

```
/* this pattern DOES NOT work! */

try {
  fetchDataAsync(processResults);
} catch (error) {
  // error handling logic
  console.log('Caught Error: ', error);
}

function fetchDataAsync(cb) { ... }

function processResults(results) {
  throw Error('something went wrong');
}
```

Application will blow up with **Uncaught Error** and *fetchData* call will not be present on stack trace

Callback Error Handling



A separate error callback or callback error argument is required to pass error details from inside an async callback

```
fetchData(processResults);

function fetchData(cb) {
  $.ajax({
    url: 'https://api.exchangeratesapi.io/latest',
    type: 'GET',
    dataType: 'json',
    success: function(results) { cb(null, results) },
    error: function(request, textStatus, httpError) { cb(httpError || textStatus) }
  });
}

function processResults(error, data) {
  if (error) {
    // error handling
  }
  // process data
}
```

Error first callbacks are a well know and established pattern

{ Exercise }

AJAX with Error Handling

← → ↻ localhost:8080

Insert amount in Eur:

33 EUR = 39.6231 USD

← → ↻ localhost:8080

Insert amount in Eur:

Error fetching rates

Callback Woes



Two very important aspects of synchronous code for function composition are:

1. They return values - feed the return value from one function straight into the next
2. They throw exceptions - if one function fails, all others in the chain are bypassed until error is handled

```
try {  
  
    var result = func3(func2(func1()));  
  
} catch (error) {  
    // handle error  
}
```

With async callbacks we have lost both aspects!

Callback Headaches



Callbacks in sequence are hard...

```
var result;

f1(function(err, data) {

    if (err) {
        // handle error
        return;
    }

    result = data;
});

var f1 = function(cb) {
    f2(function(error, result) {
        if (error) { cb(error); }
        f3(function(error, result) {
            cb(null, result);
        });
    })
};
```

In parallel they are harder...

```
var result, temporaryResults = [];

f1(function(err, result) {
    if (err) { /* process error */ return; }
    finished(result);
});

f2(function(err, result) {
    if (err) { /* process error */ return; }
    finished(result);
});

f3(function(err, result) {
    if (err) { /* process error */ return; }
    finished(result);
});

function finished(data) {
    temporaryResults.push(data);
    if (temporaryResults.length === 3) {
        result = temporaryResults[0] +
            temporaryResults[1] +
            temporaryResults[2] ;
    }
}
```

And errors so easy to lose!!!

Promises



An abstraction built on top of callbacks that gives us back functional composition and error bubbling in the async world

A Promise object represents the eventual completion (or failure) of an asynchronous operation and its resulting value

Easy in sequence...

```
var result = f1()
  .then(f2)
  .then(f3)
  .catch(function(error) {
    // handle error
  });
```

Equally as easy in parallel...

```
var result = Promise.all([f1(), f2(), f3()])
  .then(function(results) {
    return results[0] + results[1] + results[2];
  }).catch(function(error) {
    // handle error
  });
```

Un-invert the chain of responsibility:
instead of calling a passed callback, return a promise

Creating Promises



A promise can be created using a constructor function, after which it will be in one of the following states:

1. pending - initial state, neither fulfilled nor rejected
2. fulfilled - the operation completed successfully
3. rejected - the operation failed

```
var promise = new Promise(function(resolve, reject) {  
  // do some async action, promise is in pending state  
  if (/* everything ok */) {  
    resolve(value); // promise will be resolved  
  } else {  
    reject(Error(value)); // promise will be rejected  
  }  
});
```

The constructor receives an async function callback with methods for resolving or rejecting the promise

Using Promises



A promise is a *thenable* object to which callbacks can be attached

```
promise.then(function(result) {  
    // do something with the result  
}, function(err) {  
    // handle the error  
});
```

then takes two optional arguments,
a callback for success and another for failure

Once a promise has been settled, its state can not be altered, but we can always add additional handlers to it

Promise Chaining



Promises can be chained together to transform values or run additional async actions one after another

```
var promise = doSomethingAsync();

promise
  .then(doSomethingElseAsync)
  .then(doSomethingElseAsync)
  .then(doSomethingElseAsync, handleSpecificError)
  .then(doSomethingElseAsync)
  .then(doSomethingElseAsync)
  .catch(handleGenericError)
  .finally(alwaysRunThis);
```

Both specific and generic error handling callbacks can be attached to promises

Parallel Execution



Parallel promises fulfills when all of the promises have fulfilled or rejects as soon as one of the promises rejects

```
var promise = Promise.all([
  doSomethingAsync(),
  doSomethingAsync(),
  doSomethingAsync().catch(handleSpecificError,
  doSomethingAsync(),
  ]).catch(handleGenericError);
```

It fulfills with an array of the values
and rejects with the reason from the first promise that rejected

Redundant Promises



It is possible to settle a promise as soon as one of many promises settles

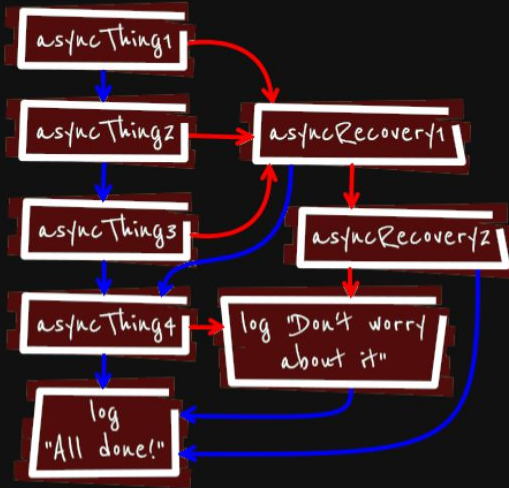
```
var promise = Promise.race([
  doSomethingAsync(),
  doSomethingAsync(),
  doSomethingAsync().catch(handleSpecificError),
  doSomethingAsync(),
]).catch(handleGenericError);
```

Used when interested only in one of the results

Advanced Async Flows



Complex async flows can be achieved with promises



```
asyncThing1().then(function() {  
  return asyncThing2();  
}).then(function() {  
  return asyncThing3();  
}).catch(function(err) {  
  return asyncRecovery1();  
}).then(function() {  
  return asyncThing4();  
}, function(err) {  
  return asyncRecovery2();  
}).catch(function(err) {  
  console.log("Don't worry about it");  
}).then(function() {  
  console.log("All done!");  
});
```

blue lines for promises that fulfill or red for ones that reject

Settled Promises



It is possible to create settled promises without actually performing any async action

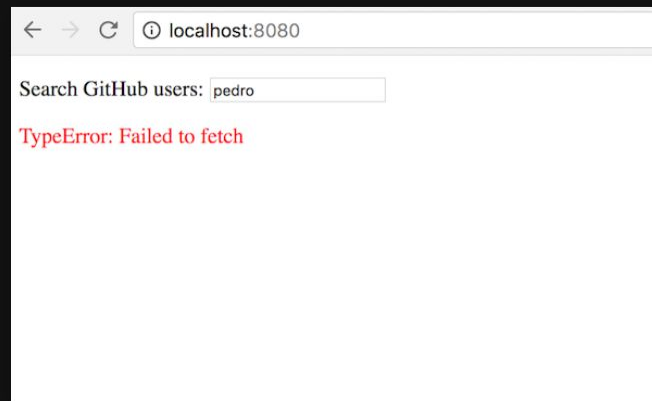
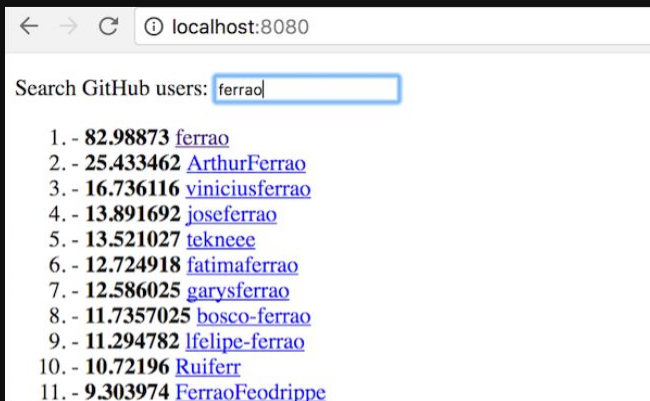
```
function doSomethingAsync() {  
  if (/* some condition */) {  
    return new Promise(asyncCallback); // do async operation  
  }  
  
  // make sure we always return a promise, even if no async code has been executed  
  
  if (/* other condition */) {  
    return Promise.resolve(value);  
  } else {  
    return Promise.reject(new Error());  
  }  
}  
  
doSomethingAsync().then(onSuccess).catch(onError);
```

Since a promise is always returned, we can use then and catch callback methods on the promise return value, which will always be executed in the next event loop tick

{ Exercise }

AJAX with Promises

Use the browser fetchapi to order a list of GitHub users by score



<https://developer.github.com/v3/search/>

{ Exercise }

✓ you should get a green check mark on all tests

```
1. ferraor@Ruis-MacBook-Pro-3: ~/Dev/Workspace/academia-codigo/js-bootcamp/exercises/04-async/js-tests (zsh)

✓ you should be able to return the decimal representation of a binary string
✓ you should be able to add with a precision of 2
✓ you should be able to multiply with a precision of 4

arrays
✓ you should be able to determine the location of an item in an array
✓ you should be able to sum the items of an array
✓ you should be able to remove all instances of an item from an array
✓ you should be able to remove all instances of an item from an array, returning the original array
✓ you should be able to add an item to the end of an array
✓ you should be able to remove the last item of an array
✓ you should be able to add an item to the beginning of an array
✓ you should be able to remove the first item of an array
✓ you should be able to join together two arrays
✓ you should be able to add an item anywhere in an array
✓ you should be able to count the occurrences of an item in an array
✓ you should be able to find duplicates in an array
✓ you should be able to square each number in an array
✓ you should be able to find all occurrences of an item in an array

strings
✓ you should be able to reduce duplicate characters to a desired minimum
✓ you should be able to wrap lines at a given number of columns, without breaking words
✓ you should be able to reverse a string
✓ you should be able to check if string is palindrome

56 passing (102ms)

~/Dev/Workspace/academia-codigo/js-bootcamp/exercises/04-async/js-tests master →
```

Single Page Application



Modern web application development paradigm made possible by AJAX and asynchronous programming



- Only one web page is requested from the server
- Each new page is generated dynamically through JavaScript DOM manipulation
- Code is typically organized in separate JavaScript modules
- A Router module allows the creation and usage of multiple page URLs without page reloading

JavaScript Modules

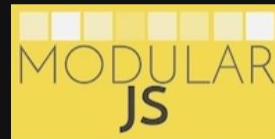


A module is a reusable piece of code that encapsulates implementation details and exposes public methods so it can be used by other code

Modules should allow us to:

- abstract code
- encapsulate code
- code reuse
- manage code dependencies

Modular JavaScript



Before ES6, no official syntax to define modules existed and developers were often times forced to choose between:

- code their entire application in a single file, which results in a big mess
- expose on the global scope some variation of the module pattern which often lead to namespace collisions

```
<!--  
  developers used to write a bunch of script tags with implicit dependencies that had to be  
  manually ordered and pray for no ordering issues or collisions on the global scope...  
-->  
<script src="https://cdnjs.cloudflare.com/ajax/libs/modernizr/2.8.3/modernizr.min.js"></script>  
<script src="https://code.jquery.com/jquery-3.2.1.min.js"></script>  
<script src="https://maxcdn.bootstrapcdn.com/bootstrap/4.0.0/js/bootstrap.min.js"></script>  
<script src="https://www.google-analytics.com/analytics.js"></script>  
<script src="js/main.js"></script>  
<script src="js/module1.js"></script>  
<script src="js/module2.js"></script>  
<script src="js/module3.js"></script>
```

Module Systems



{ EXPORT }

{ IMPORT }

Various formats to define modules in JavaScript were developed, minimizing the lack of native JavaScript module syntax

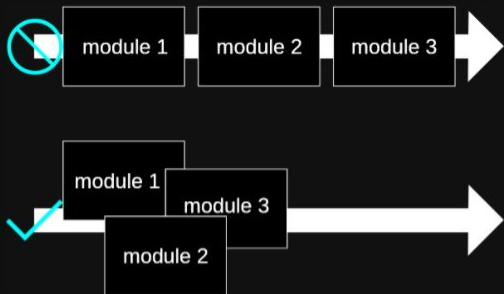
Some of the most widely adapted and well known formats are:

- AMD - Asynchronous Module Definition, an asynchronous module system for the browser
- CommonJS - Popular system used by Node, can be used in the browser with tools such as Browserify or Webpack
- UMD - Universal Module Definition, supports both AMD and CommonJS modules, both in Node and the Browser
- ES6 modules - native javascript modules, supported on newer browsers only

Asynchronous Module Definition (AMD)



A mechanism for defining and asynchronously loading modules and their respective dependencies for the browser environment



```
// Filename: foo.js
```

```
// Calling define with a dependency array and a factory function  
define(['jquery'], function($) {
```

```
    // Methods
```

```
    function myFunc() {
```

```
        // do something with jquery here...
```

```
    };
```

```
    // Exposed public methods
```

```
    return myFunc;
```

```
});
```

RequireJS



JavaScript file and module loader supporting the AMD standard

A JavaScript application consisting of multiple modules and their respective dependencies can be bootstrapped with a single script tag:

```
<!DOCTYPE html>
<html>
  <head>
    <title>My Web App</title>

    <!-- data-main attribute tells require.js to load the js/app.js file after require.js loads -->
    <script data-main="js/app"
      src="https://cdnjs.cloudflare.com/ajax/libs/require.js/2.3.5/require.min.js"></script>
  </head>
  <body>
    <h1>My Web App</h1>
  </body>
</html>
```

RequireJS Configuration



A global require object is exposed, which can be used to configure RequireJS and bootstrap the application

```
// Filename: js/app.js
require.config({

    // path to fetch all our modules from
    baseUrl: '/js/app'
});

// set the main module as our application entry point
require(['main'], function() {
    console.log('All modules finished loading');
});

// Filename: js/main.js
require(['router'], function(router) {

    $(document).ready(function() {
        console.log('dom is ready');
        router.start();
    });
});
```

Client Side Routing



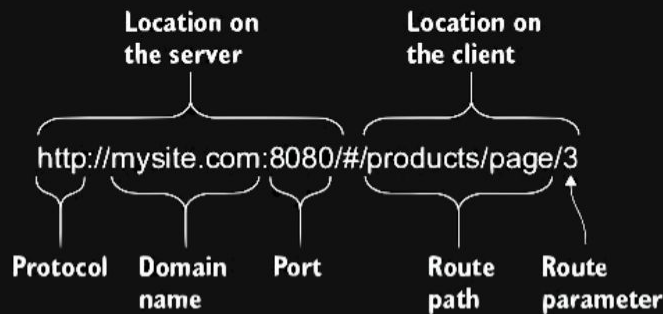
The SPA model breaks the browser page history navigation using the Forward/Back buttons

Both **Hash-based routing** and the **HTML5 History API** enable modifications to the page URL without reloading the page, allowing creation of separate URLs for different views

Hash-based Routing



URLs can contain some data prepended with a # character
This data is called the hash fragment



Changes in the hash fragment never trigger a page reload and can be used for storing the state of the client application

{ Hands On }

Simple JavaScript Router

{ Exercise }

MarvelApp

